

Few-shot Learning & Meta Learning

Updated 12-2-24, **12-8-25**

CJ Chung

“Few-shot” Learning (FSL)

- A machine learning technique that uses a small number of labeled examples to learn how to make accurate predictions.
- Few-shot learning is often used for classification tasks when there isn't enough training data. The success of few-shot learning is measured using a *metric* that's appropriate for the specific goal.
- In the previous XOR example, actually we did **one**-shot learning. Only one sample for each class using ES worked well!!

Some characteristics of Few-Shot (FS) models

- Prior knowledge: Few-shot models use prior knowledge and learning experiences to adapt to new tasks.
- Generalization: Few-shot models can generalize new data based on a small number of examples.
- Task-specific: Few-shot models can adapt their pre-trained knowledge to a specific task context.
- Meta-learning: Few-shot learning is part of the meta-learning paradigm, which means learning to learn *fast*

What is the difference between FSL and MetaL

- Few-Shot Learning transfers knowledge within a specific task. It focuses on how a model can generalize from a few examples to make accurate predictions within that single task.
- Meta Learning, on the other hand, transfers knowledge across multiple tasks. (See next slide)

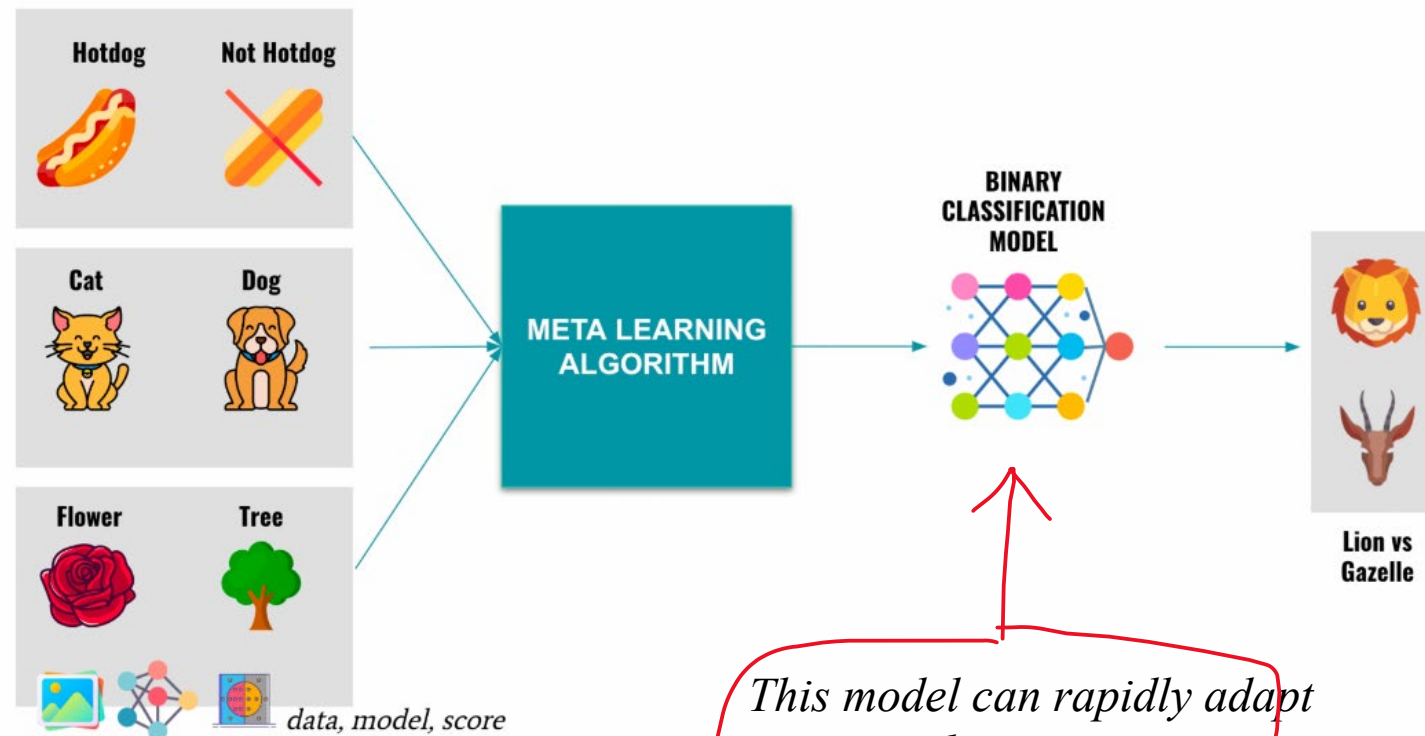
Meta Learning (Learning to learn)

The process of learning how to learn, often used in ML to enable a model to adapt to new tasks more quickly by using experience from previous, *related* tasks.

- Meta-learning systems are trained on a collection of different tasks to extract general information about the learning process.
- This enables the model to learn new, unseen tasks much *faster* and with *fewer* examples than it would with traditional methods.
- This includes techniques like learning initial model parameters that work well across many tasks or learning an optimization process that can be quickly adapted to new tasks

Meta Learning - Introduction to Meta Learning and Neural Architecture Search

- <https://www.thinkautonomous.ai/blog/meta-learning/>



Model-Agnostic Meta-Learning (MAML)

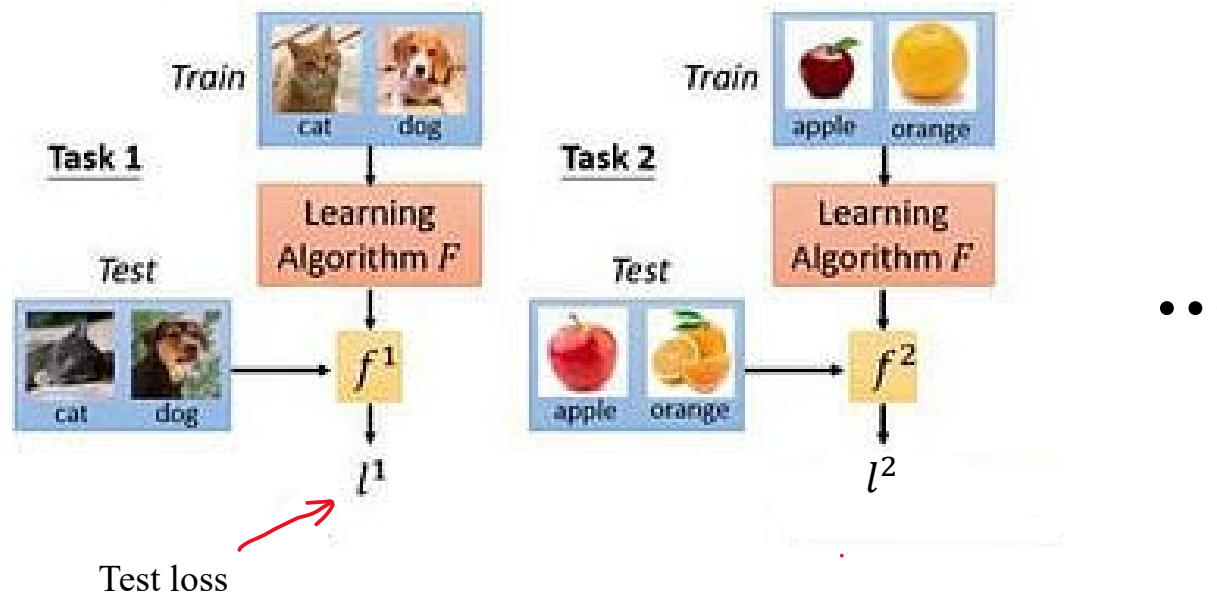
To minimize

Meta Learning $L(F) = \sum_{n=1}^N l^n$

• Defining the goodness of a function F

N tasks

Testing loss for task n after training



Review: GD algorithm concept, when full-batch size is n

$\hat{Y} = f_W(X)$ Model f_W 's prediction with input X

$Y =$ Desired target output values

$$L = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

$$W' = W - \alpha \frac{\partial L}{\partial W} \quad \equiv \quad W' = W - \alpha \nabla L(W)$$

x0	x1	y
0	0	0
0	1	0
1	0	0
1	1	1

For example, $n = 4$
for AND logic

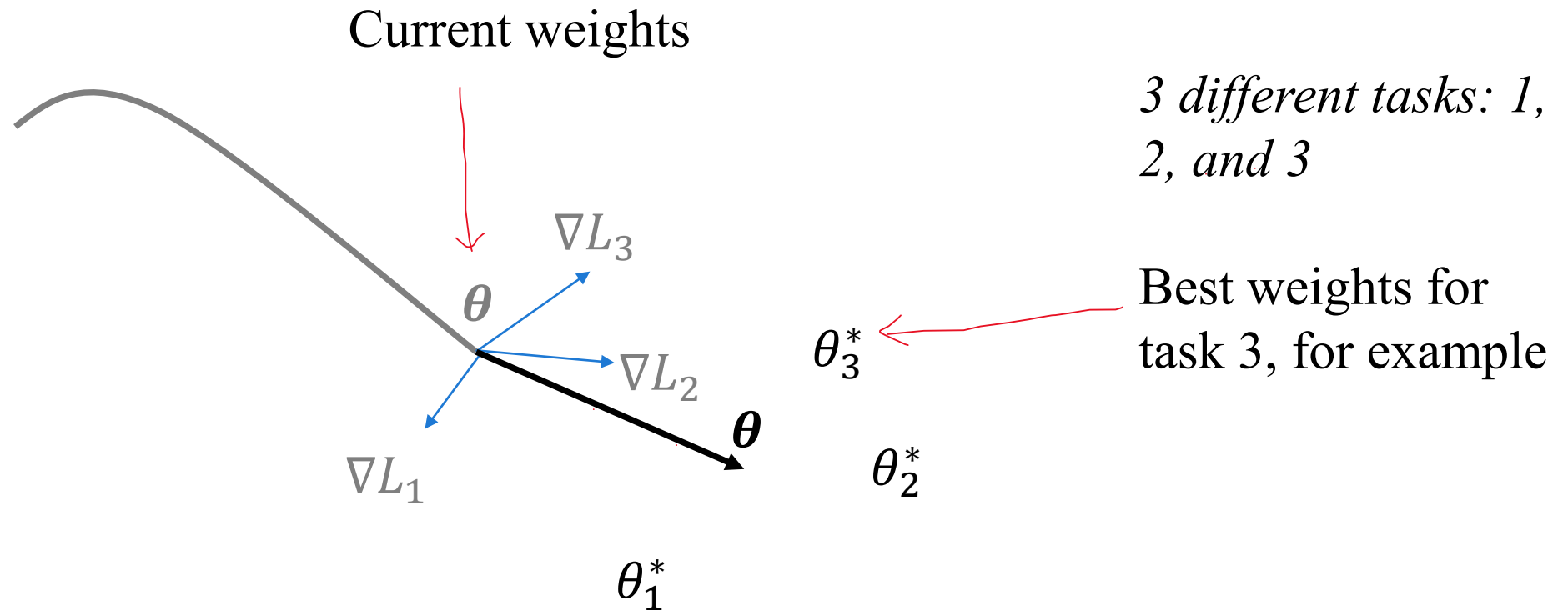
MAML (Model-Agnostic Meta-Learning)

notations

- Model f parameterized by θ (weights)
- Model f maps x to y
- T : task
- $p(T)$: task distribution
- K -shot learning, K samples for each task
- L : loss function

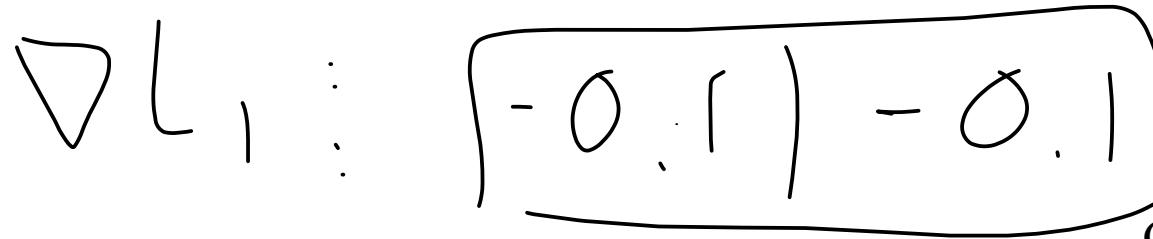
Model-Agnostic Meta-Learning (MAML) Intuition

Assume 2D space (2 weights)

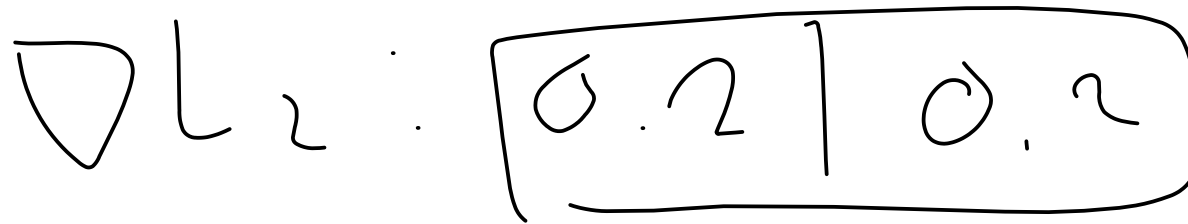
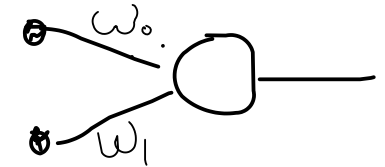


$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$$

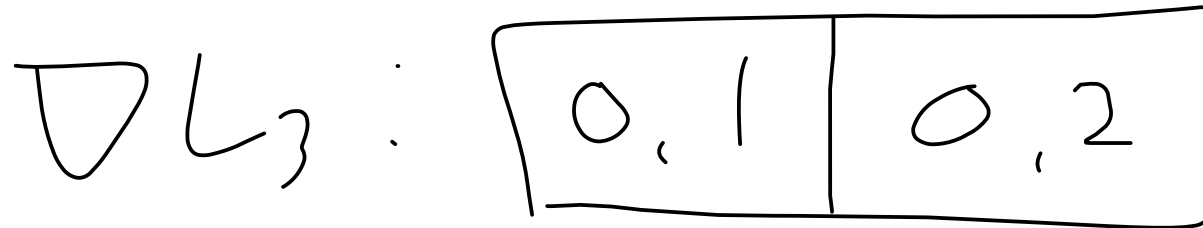
The goal is to optimize weight vector θ that can rapidly adapt to new tasks.

Good weights for a generic task at t
 $\cdot 2 - 0.2$ $2 - 0.3$
Good weights for tasks at $t+1$ 

Gradient change for task 1



Gradient change for task 2



Gradient change for task 3

$\sum \nabla L_m :$
0.2
0.3

Meta weight gradients

MAML algorithm

Algorithm 1 Model-Agnostic Meta-Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

- 1: randomly initialize θ
- 2: **while** not done **do**
- 3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
- 4: **for all** $\mathcal{T}_i$ **do**
- 5: Evaluate $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ with respect to K examples
- 6: Compute adapted parameters with gradient descent: $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$
- 7: **end for**
- 8: Update $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$
- 9: **end while**

```

for epoch in range(num_epochs):
    # init a list of arrays with same shapes as trainable weights of a model, but filled with zeros
    meta_weights_gradients = [np.zeros_like(weight) for weight in model.get_weights()]
    # task training
    for _ in range(num_tasks):
        # Generate task-specific data
        X_train, y_train = generate_data(num_samples=10) # was 20
        if epoch == 0:
            plot_data(X_train, y_train)

        # Perform task-specific training and get the adapted weights
        adapted_weights = single_task_train(model, X_train, y_train, task_lr, num_inner_steps) # single_task_train

        # Compute the gradient with respect to the original model's weights
        # with as: exception handling to make the code cleaner and much more readable than try
        with tf.GradientTape() as tape:
            model.set_weights(adapted_weights)
            y_pred = model(X_train)
            y_train = y_train.reshape(-1, 1) # Reshape y_train to match y_pred's shape
            task_loss = tf.keras.losses.binary_crossentropy(y_train, y_pred)

        # Accumulate gradients for meta-update
        grads = tape.gradient(task_loss, model.trainable_weights)
        meta_weights_gradients = [meta_grad + grad for meta_grad, grad in zip(meta_weights_gradients, grads)]

    # Meta-update: Apply the averaged gradients
    optimizer.apply_gradients(zip(meta_weights_gradients, model.trainable_weights)) # apply_gradients(): used
    print(f"Epoch {epoch + 1}/{num_epochs}, Meta-Loss: {np.mean(task_loss)}") # Print loss to track progress

```

meta-learning.ipynb

<https://arxiv.org/abs/1703.03400>

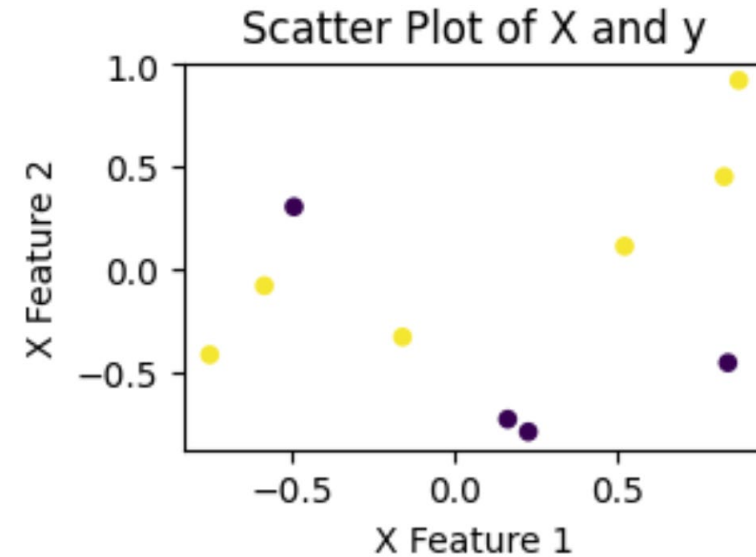
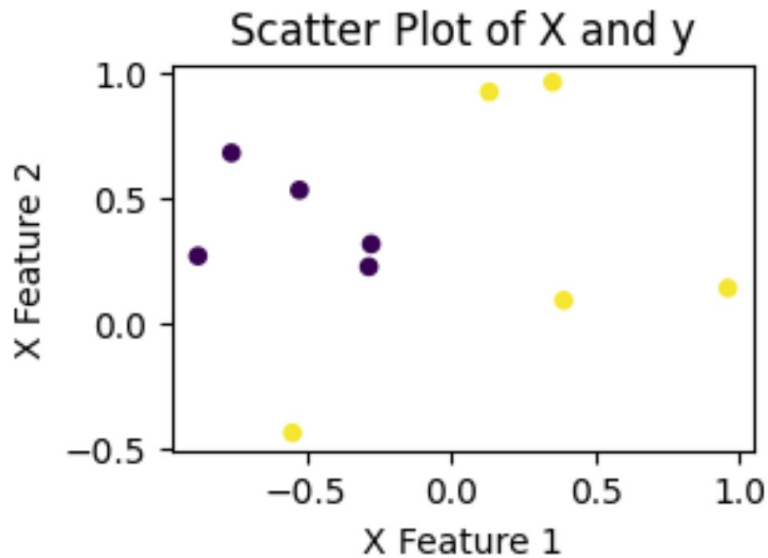
<https://medium.com/@jasperorachen/model-agnostic-meta-learning-for-fast-adaptation-of-deep-networks-maml-e75d1cfcca6>

MAML example code: **meta-learning.ipynb**

```
def generate_data(num_samples=100):
    X = np.random.rand(num_samples, 2) * 2 - 1 # [-1,1]
    y = (X[:, 0] * X[:, 1] > 0).astype(int) # "1" if the prod is greater than 0
                                           # and "0" otherwise

    return X, y
```

```
(20, 2)
[[ 0.07595735  0.9775055 ]
 [-0.86397759  0.87978316]
 [-0.10166     0.24779108]
 [-0.0407434  -0.23818639]
 [ 0.00128488  0.25556238]
 [ 0.16431864  -0.62262768]
 [ 0.15489753  0.65275415]
 [ 0.01069599  -0.04283957]
 [-0.58703918  0.85886016]
 [ 0.69026243  0.91393303]
 [-0.07756483  -0.17029094]
 [-0.14945476  -0.00594775]
 [ 0.98161452  0.78370499]
 [ 0.96722397  0.41378343]
 [ 0.17115002  -0.07810136]
 [-0.39318513  0.1552766 ]
 [-0.88575908  -0.61956377]
 [ 0.35919749  -0.35508424]
 [-0.96808219  0.77285538]
 [-0.96532626  0.87184501]]
```

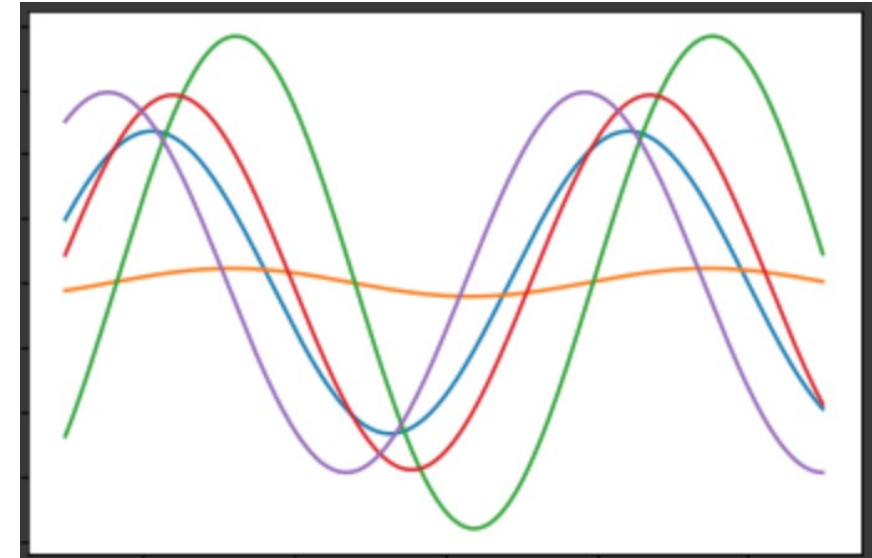


...

Meta-Learning in Practice 2 — Train a Simple Model Learn to Learn

The task is to use a deep neural network to predict output of the following function with a **random** pair of a and b :

$$y = a * \sin(x + b)$$



Some videos on MetaL

- <https://youtu.be/43OkKyFvd4E?si=x80tIAKWcEITH8eA>
- <https://youtu.be/mftNApIM1Yc?si=qaAgBecytJymRHRL>
- <https://youtu.be/lkDw22a8BDE?si=9mb4CGAZl2T9npvt>
- <https://www.youtube.com/watch?v=u5BkO8XMS2I>
- <https://youtu.be/wT45v8sIMDM?si=XLs03SMqisRZ6zqt>
- ...

MetaL and EC are related

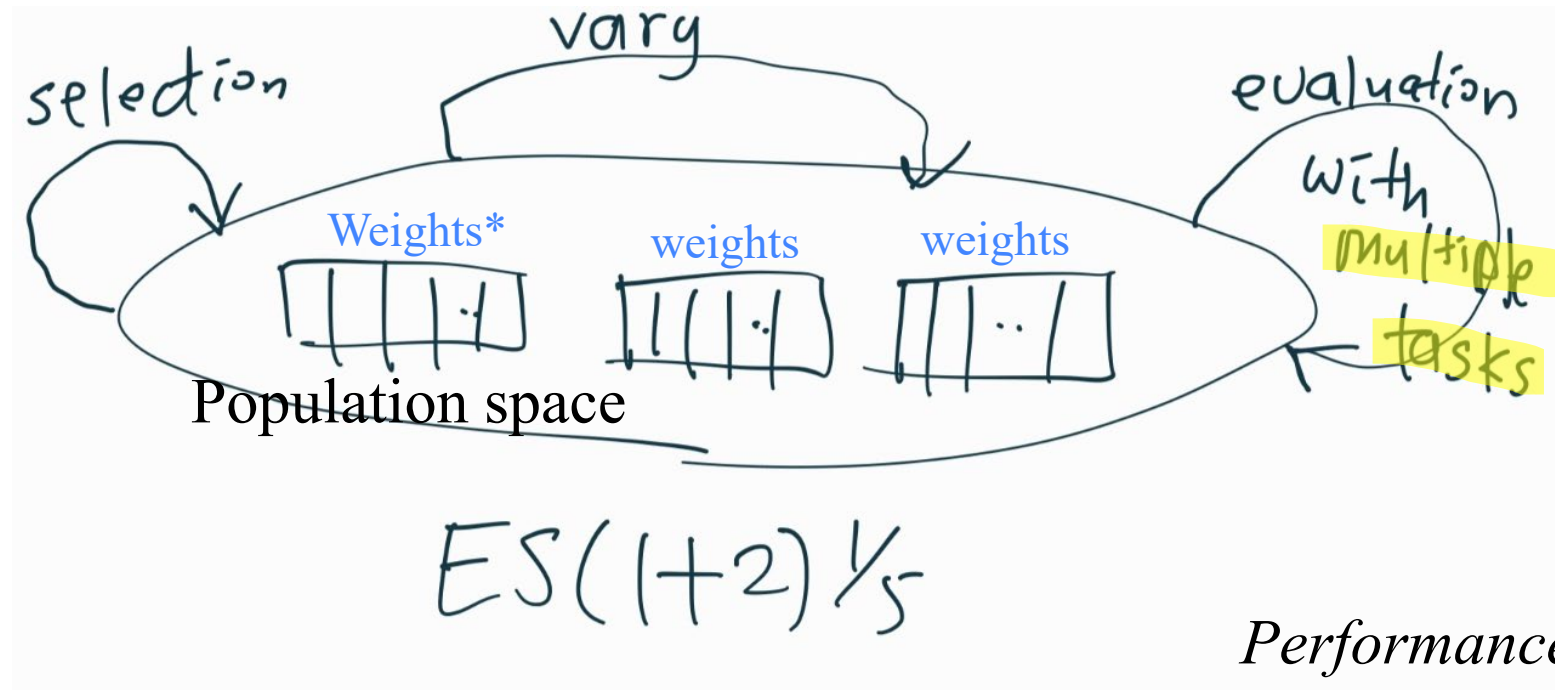
- Make Meta-Learning Stronger through Evolution
<https://medium.com/geekculture/make-meta-learning-stronger-through-evolution-166139f54360>
- Population-Based Evolution Optimizes a Meta-Learning Objective
<https://arxiv.org/abs/2103.06435>
- Meta-learning, or "learning to learn," and evolutionary computation (EC) can be combined to create algorithms that learn to optimize their own learning processes, leading to faster and more efficient adaptation to new tasks.

Review: Evolutionary Weight Optimiztaion without BP

- XOR
- XOR3

Research idea using ES (instead of GD) for meta-learning for multiple tasks (without using BP)

Assume Model Architecture is fixed

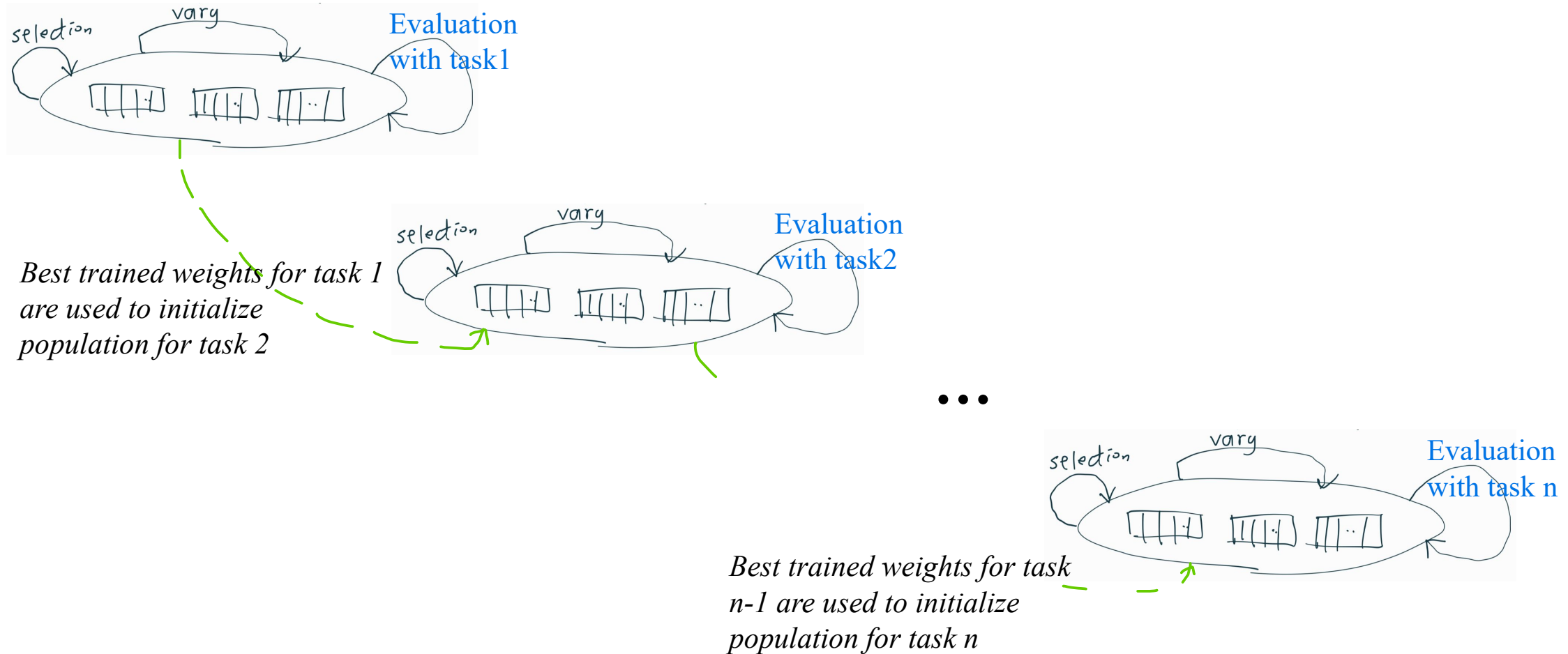


(*) All or partial weights of the model.

Performance metric based termination condition

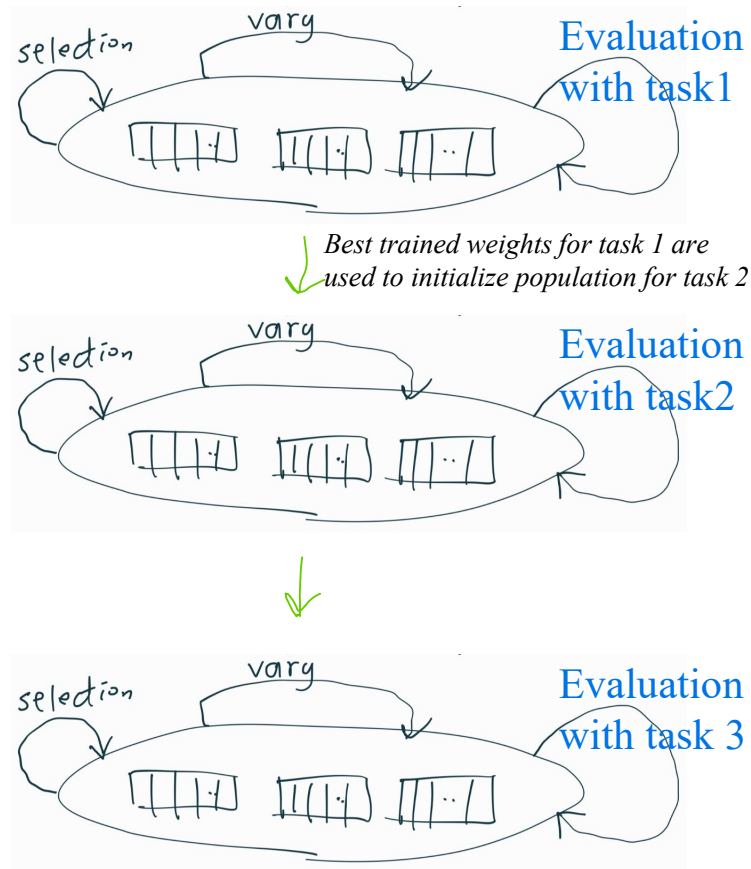
Research idea using ES for meta-learning for multiple tasks (without using BP)

Model Architecture is fixed



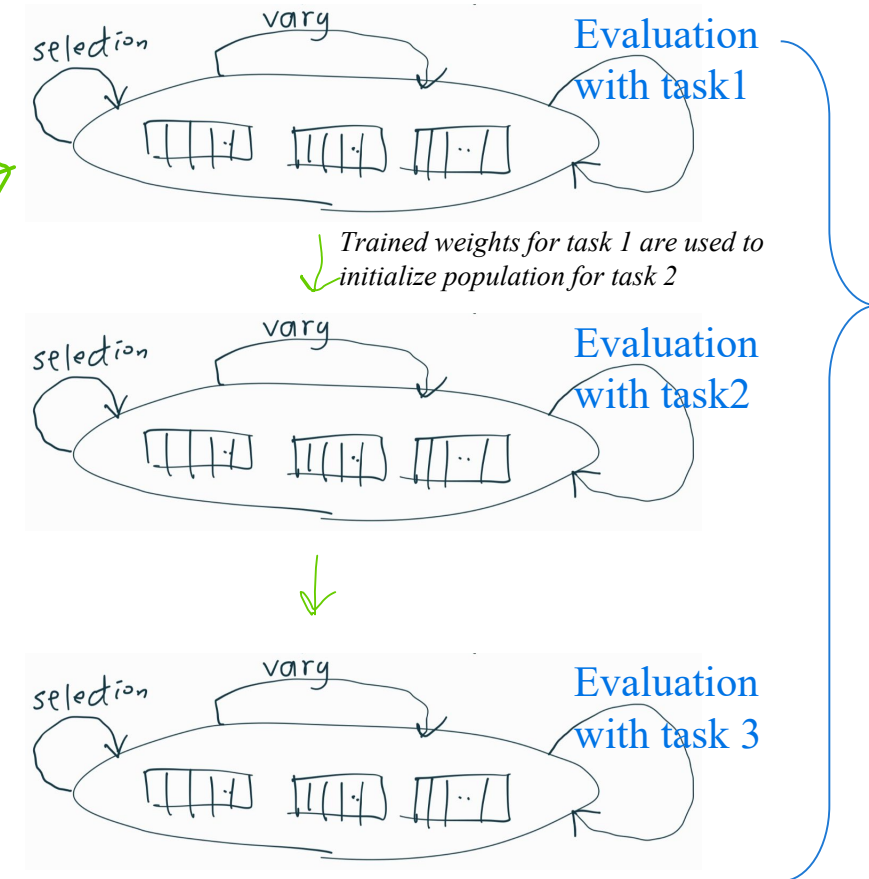
Research idea using ES for meta-learning for 3 tasks with multiple “phases”

Model Architecture is fixed



Phase 1

Summed weights from task 1,2,3 are used to initialize population for task 1 in the next phase

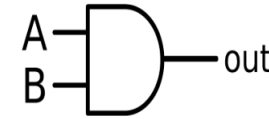


Phase 2

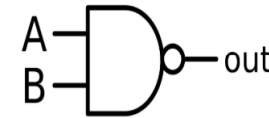
...

A Simple Research Problem I

- Task2: AND logic
- Task4: NAND logic



Input		Output
A	B	$Y=A.B$
0	0	0
0	1	0
1	0	0
1	1	1



Input		Output
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

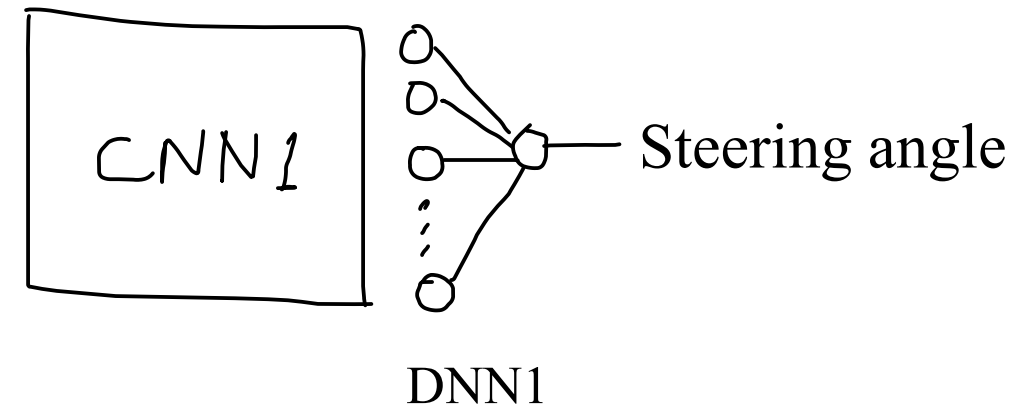
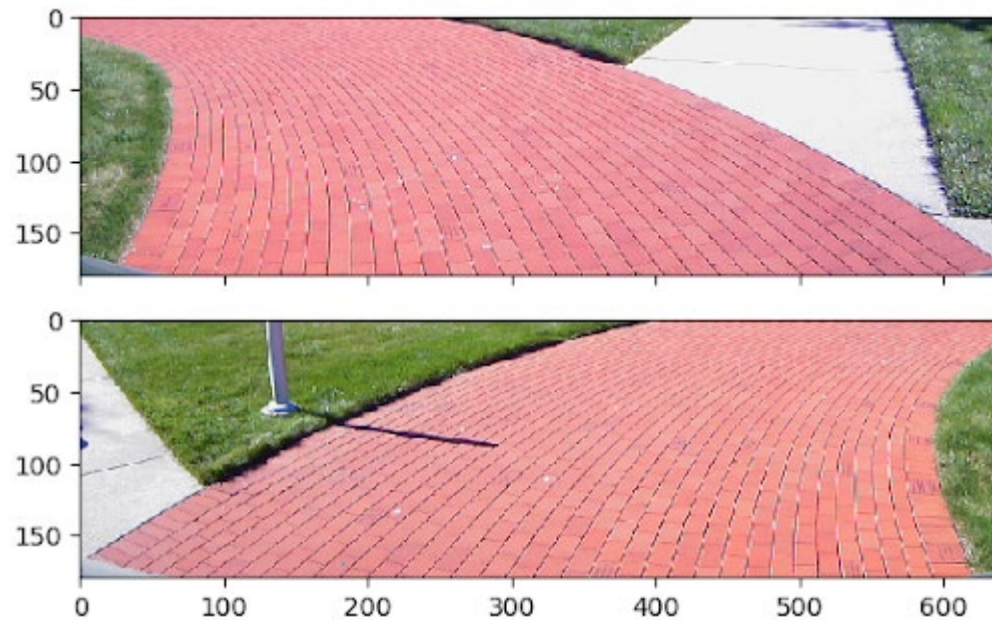
- Assume we have a NN for AND. Can we develop model for NAND by modifying just some weight values of the AND NN?

A Simple Research Problem II

- Task1: OR logic
 - Task2: AND logic
 - Task3: NOR logic
 - Task4: NAND logic
-
- How can we train XOR logic *fast* when we have best models for the above problems? (we have best weight values for the above)

An Application: To drive 3 roads (Circle, Straight, and Curved)

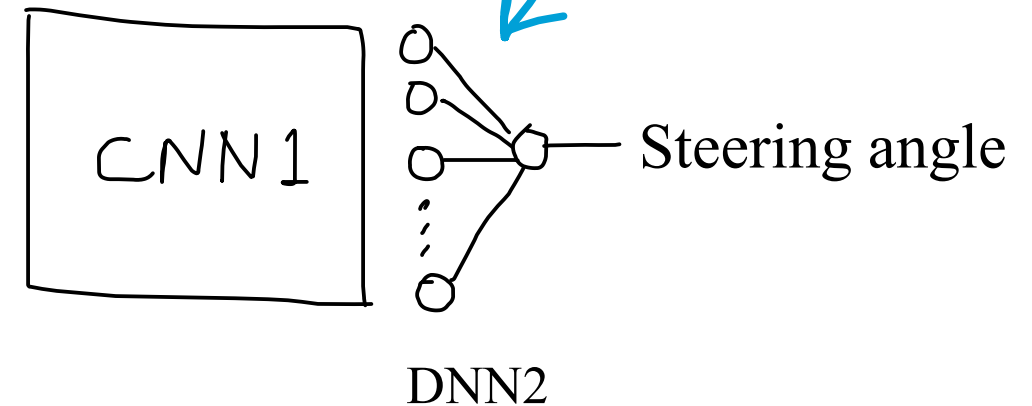
Circle drive



Straight road



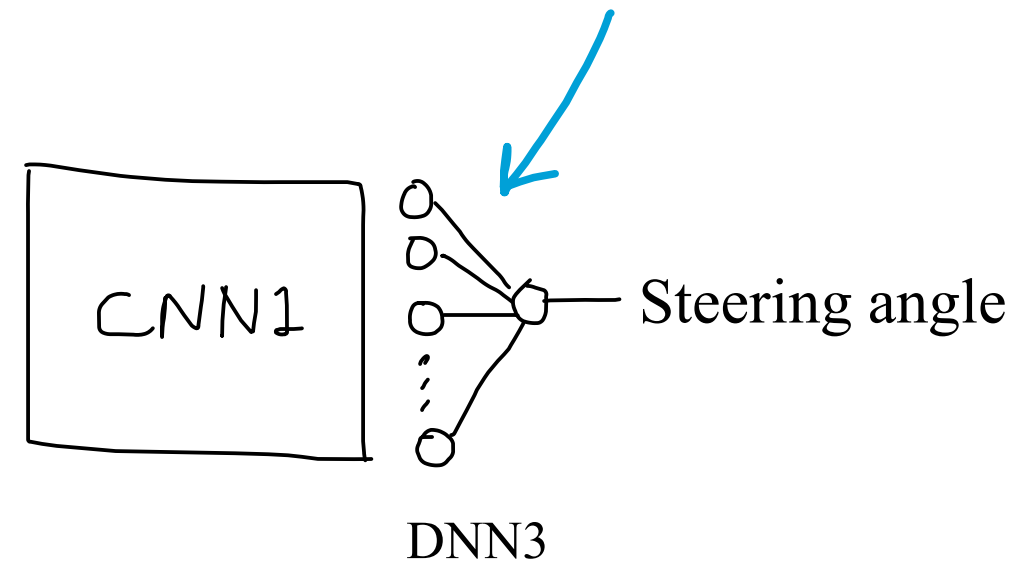
Weight optimization using ES. Start evolution with weights found from circle drive (DNN1)



Straight & curved road



Weight optimization using ES. Start evolution with weights found from the straight road (DNN2)



What's next?